

Customer personality analysis: segmenting and predicting behavior

Ciaboco Alessandro
Matricola: 68530A

May 2026

Abstract

The goal here is to optimize the marketing budget by finding high-value customers. By looking at old retail data, I found buying patterns to make a predictive model. First I used pca and k-means clustering to split the customers into different economic groups. Then I trained and checked a few models (logistic regression, lasso, random forest, ann) to predict how customers will react to a new campaign. In the end, the models show we can cut costs by only targeting people who are likely to buy.

Dataset structure

The data gives a good look at our customers. I split the variables into four main groups:

- Demographics: basic info like age (from birth year), income, relationship status, and dependents.
- Product spending: total spent over two years on things like wine, meat, and sweets. I put these together into a TotalSpent variable to see overall buying power.
- Channel engagement: metrics on how people interact with the store, comparing web visits to real purchases in-store or from a catalog.
- Campaign history: past records of who accepted old marketing offers. The main target variable to predict is ResponseTarget (if they accepted the current campaign).

Data cleaning

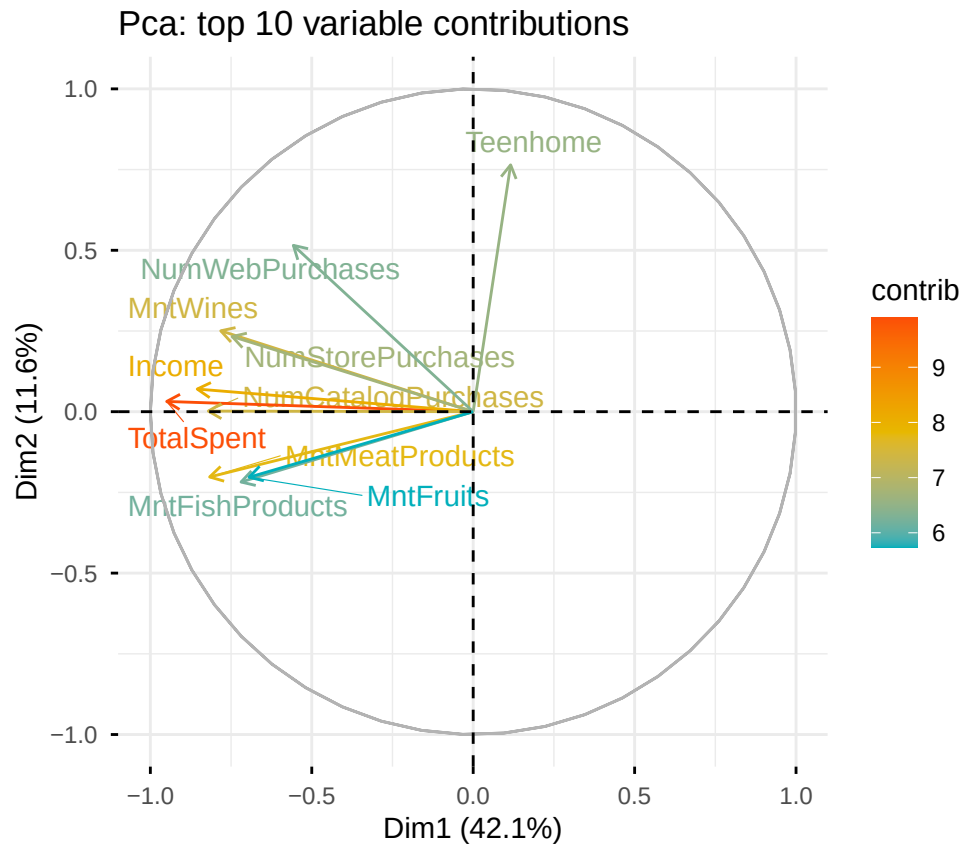
Data cleaning was needed to make the model work better and remove bias:

1. Missing data: rows without an income value were removed. Guessing salaries might add bias, so deleting them is safer.
2. Extreme outliers: some records had birth years like 1893, and a few incomes were over 600,000. I capped age at 100 and income at 600k so they don't mess up the model.
3. Zero-variance features: the variables Z_CostContact and Z_Revenue had the same value in every row. Since they don't help predict anything, I dropped them.
4. Administrative variables: customer ID and join dates were dropped so the algorithms don't find fake links.

Part 1: unsupervised learning

Principal component analysis (pca)

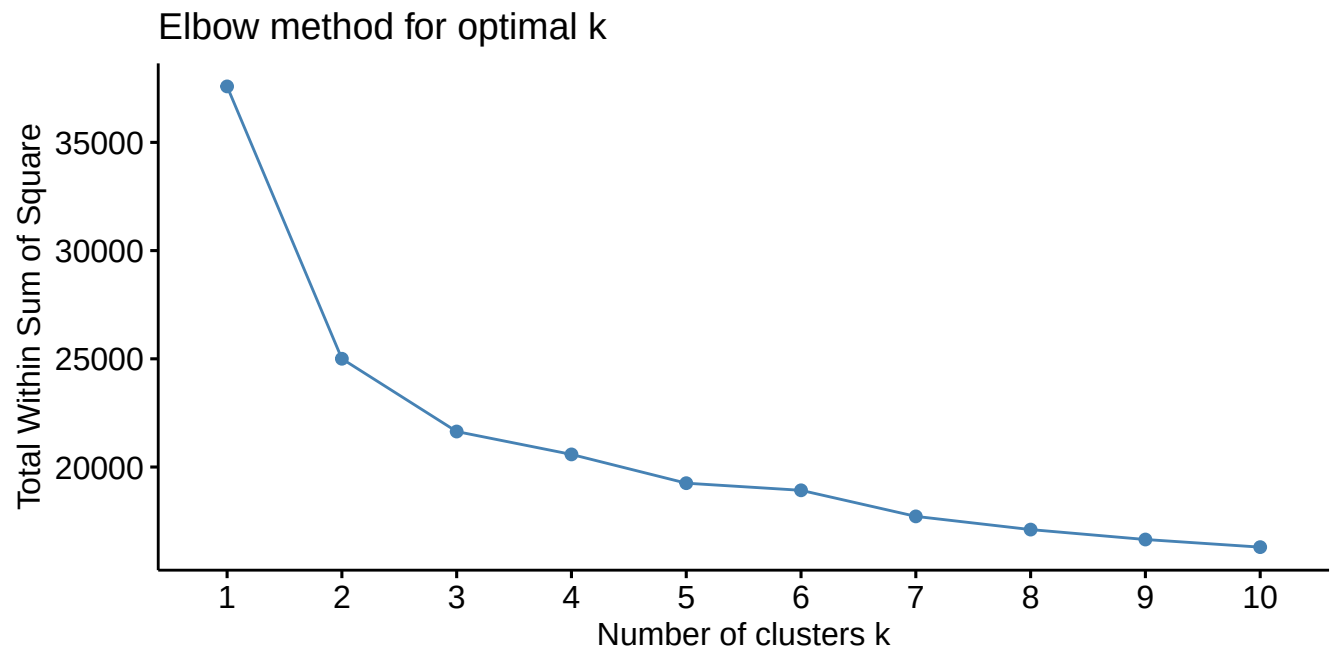
Since there's a lot of data, I ran a pca to shrink the features and see what causes the most variance.



The biplot for the first two components shows a clear split. Money variables (income, totalspent, meat, and wine) group together, pushing variance one way. On the other hand, NumWebVisitsMonth goes the opposite way. This means there's an inverse link: the people who visit the site most are often the ones who spend the least.

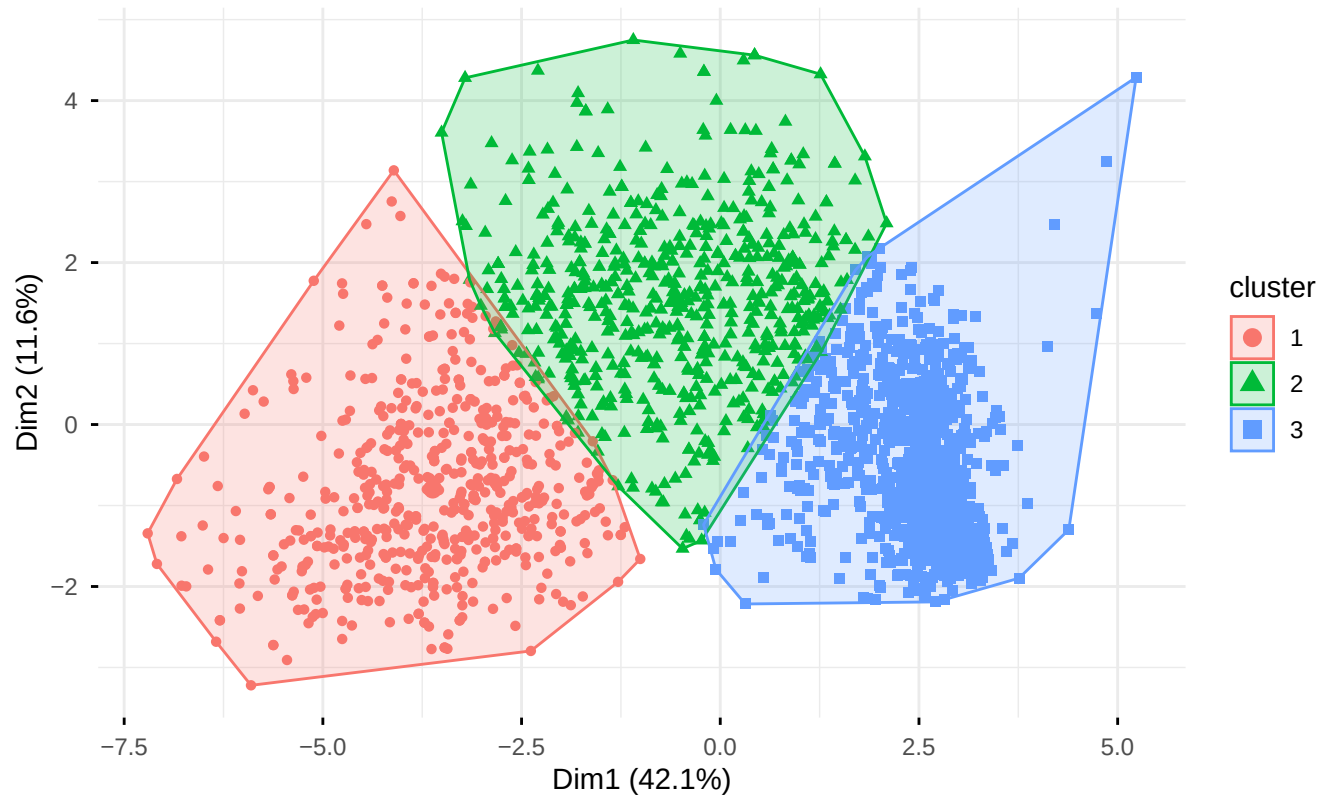
Clustering analysis

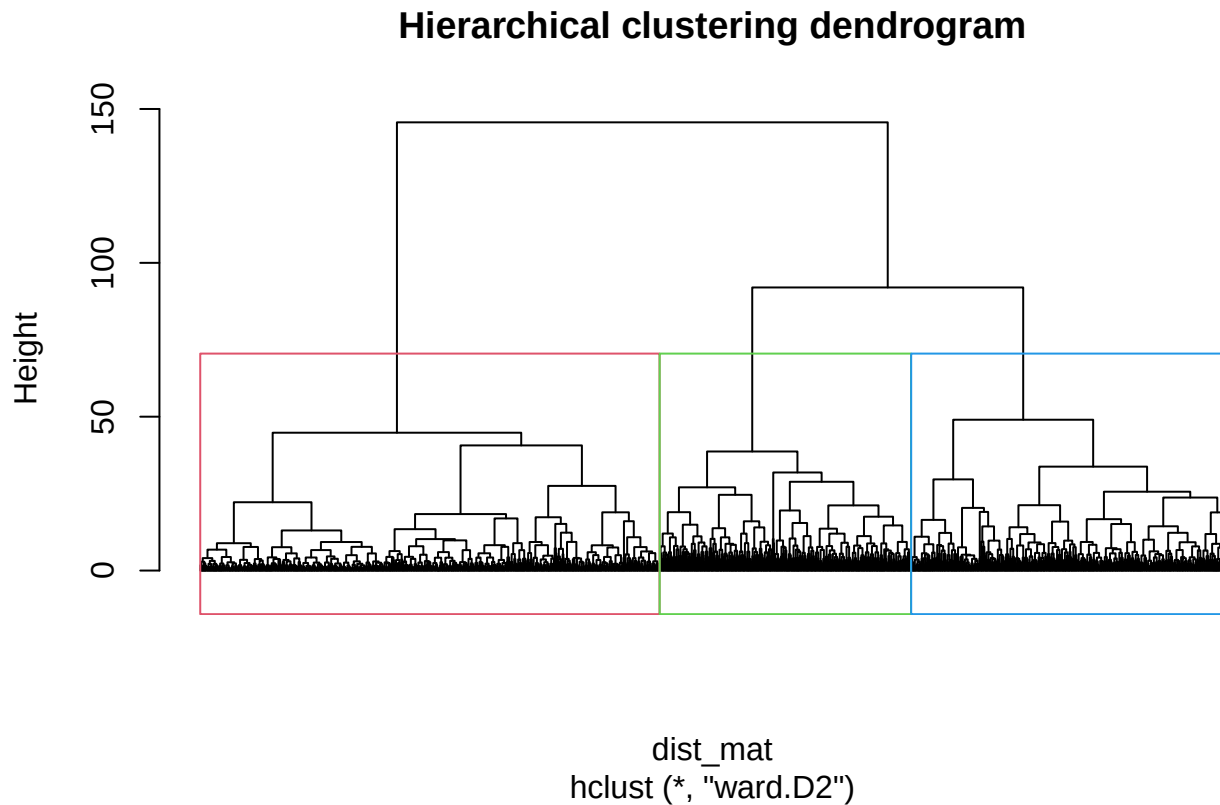
To find the best number of customer segments, I checked the within-cluster sum of squares (wss) elbow curve.



The elbow curve points to $k=3$ as the sweet spot. I ran k-means with three centers, and to be sure it wasn't a fluke, I checked the results against a hierarchical dendrogram (ward's method).

K-means clustering (k=3)





Both clustering methods match up well. The k-means plot shows three clear groups, and the dendrogram splits into three main branches. The customers break down into three tiers: high-value buyers driving premium sales, an average middle tier, and a big group of users who visit a lot but spend little.

The table below puts actual numbers behind these labels.

Table 1: Cluster profiles: average values by segment

Cluster	N	Avg_Income	Avg_Spent	Avg_Recency	Avg_WebVisits
1	583	75880	1389	50.0	2.9
2	614	57907	711	47.8	5.7
3	1015	34620	96	49.2	6.5

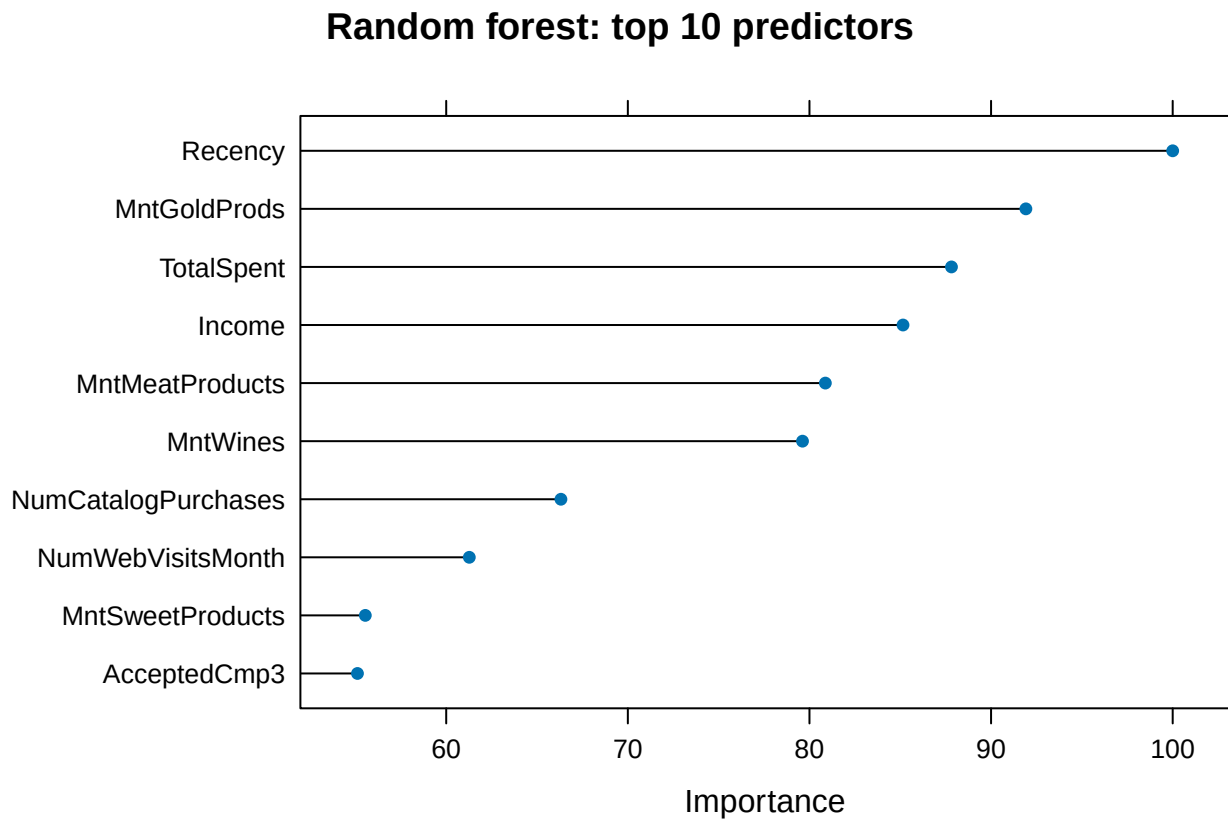
The numbers confirm the structure. The cluster with the highest income and total spent is the premium tier, the one with the most web visits and the lowest spending is the browser tier, and the middle cluster sits in between on all metrics.

Part 2: supervised learning

For the predictive models, the dataset was split 80/20 into train and test sets. I used 5-fold cross-validation on the training data to stop the models from overfitting.

Feature importance

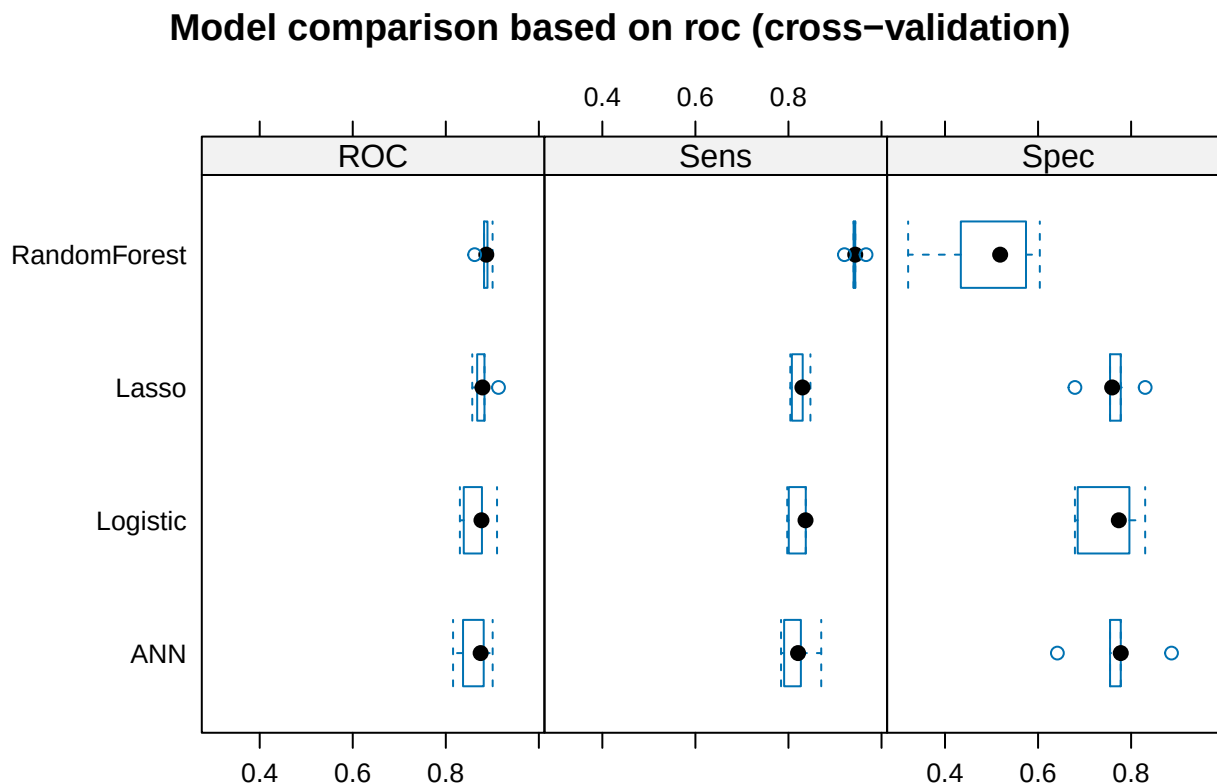
I pulled the feature importance from the random forest model to see which variables matter most for predicting.



The results are pretty clear. Recency (days since last purchase) is the strongest sign someone will accept the campaign. Past campaign actions also rank high. On the flip side, basic demographics like age don't matter much. In this dataset, how people behave matters way more than demographic info.

Model comparison

The four models were checked based on their area under the roc curve (auc) during cross-validation.



The random forest model does the best, showing the highest median accuracy and more stability than the neural network. The linear models (logistic regression, lasso) are worse, which hints at non-linear habits that simple linear equations miss.

Test set evaluation

To see how it works in the real world, I tested the random forest model against the 20% holdout test set.

```
##           Reference
## Prediction  No  Yes
##           No 362  32
##           Yes  13  34
```

Even though overall accuracy is high, the confusion matrix shows a common issue with unbalanced data. The model is a bit too careful: it's great at spotting customers who will ignore the campaign (high specificity), but it misses quite a few actual positive responses (low sensitivity).

Since the cost of sending a digital offer is basically zero, it makes sense for the business to drop the probability threshold below 0.5. This catches more potential conversions, trading a small bump in false positives for better overall engagement.


```
##           Reference
## Prediction  No  Yes
##           No 295  12
##           Yes  80  54
```

```
##
## Sensitivity at threshold 0.3: 0.818
```

```
##
## Specificity at threshold 0.3: 0.787
```

Dropping the threshold to 0.3 picks up a bigger share of real positive responses. Specificity goes down a bit, but in this context that's fine — the extra false positives just mean sending an offer to a few more people who won't convert, which costs almost nothing.

Conclusions

Using the same marketing approach for everyone is a waste of time. The clustering shows three different spending groups, so we need different strategies. Plus, the random forest model proves that recent engagement beats demographic guesses. By running contact lists through this model—focusing on recency and past actions—marketing teams can drop acquisition costs and put their effort into the groups most likely to convert.

Appendix: r code

```
knitr::opts_chunk$set(echo=FALSE,warning=FALSE,message=FALSE,fig.align='center',fig.width=7,fig.height=7)

library(dplyr)
library(ggplot2)
library(factoextra)
library(caret)
library(glmnet)
library(randomForest)
library(nnet)

# Data loading and cleaning
df<-read.csv("marketing_campaign.csv",sep="\t",stringsAsFactors=TRUE)
df<-df%>%mutate(Age=2026-Year_Birth,TotalSpent=MntWines+MntFruits+MntMeatProducts+MntFishProducts+MntSweets+MntVegs+MntGrains)

df_clean<-df%>%select(-ID,-Z_CostContact,-Z_Revenue,-Year_Birth,-Dt_Customer,-Response)

num_vars<-df_clean%>%select_if(is.numeric)%>%select(-starts_with("Accepted"),-Complain)
scaled_data<-scale(num_vars)
pca_res<-prcomp(scaled_data,center=FALSE,scale.=FALSE)
fviz_pca_var(pca_res,col.var="contrib",gradient.cols=c("#00AFBB","#E7B800","#FC4E07"),repel=TRUE,select.var=Function)
fviz_nbclust(scaled_data,kmeans,method="wss")+labs(title="Elbow method for optimal k")
# K-means analysis
set.seed(123)
km_res<-kmeans(scaled_data,centers=3,nstart=25)
fviz_cluster(km_res,data=scaled_data,geom="point",ellipse.type="convex",ggtheme=theme_minimal())+labs(title="K-means clustering")

dist_mat<-dist(scaled_data,method="euclidean")
hc_res<-hclust(dist_mat,method="ward.D2")
plot(hc_res,labels=FALSE,hang=-1,main="Hierarchical clustering dendrogram")
rect.hclust(hc_res,k=3,border=2:4)
# Cluster profiling: average values per segment
df_clean %>%
  mutate(Cluster = as.factor(km_res$cluster)) %>%
  group_by(Cluster) %>%
  summarise(
    N = n(),
    Avg_Income = round(mean(Income), 0),
    Avg_Spent = round(mean(TotalSpent), 0),
    Avg_Recency = round(mean(Recency), 1),
    Avg_WebVisits = round(mean(NumWebVisitsMonth), 1)
  ) %>%
  knitr::kable(caption = "Cluster profiles: average values by segment")
set.seed(123)
trainIndex <- createDataPartition(df_clean$ResponseTarget, p=0.8, list=FALSE)

# forzatura a data.frame puro per aggirare il bug di caret sull'upsampling
train_data <- as.data.frame(df_clean[trainIndex,])
test_data <- as.data.frame(df_clean[-trainIndex,])

cv_control <- trainControl(method="cv", number=5, classProbs=TRUE, summaryFunction=twoClassSummary, sampleSize=1000)
```

```

set.seed(123)
log_model <- train(ResponseTarget~., data=train_data, method="glm", family="binomial", metric="ROC", tr

set.seed(123)
# CORREZIONE sintassi: preProcess = c(...)
lasso_model <- train(ResponseTarget~., data=train_data, method="glmnet", preProcess = c("center","scale

set.seed(123)
rf_model <- train(ResponseTarget~., data=train_data, method="rf", metric="ROC", trControl=cv_control, t

set.seed(123)
ann_model <- train(ResponseTarget~., data=train_data, method="nnet", preProcess = c("center","scale"), n
# Var imp
plot(varImp(rf_model),top=10,main="Random forest: top 10 predictors")
# Model compare
results<-resamples(list(Logistic=log_model,Lasso=lasso_model,RandomForest=rf_model,ANN=ann_model))
bwplot(results,main="Model comparison based on roc (cross-validation)")
# Test set evaluation at default threshold 0.5
pred_rf<-predict(rf_model,newdata=test_data)
conf_mat<-confusionMatrix(pred_rf,test_data$ResponseTarget,positive="Yes")
conf_mat$table
# Lower threshold to 0.3 to improve sensitivity
prob_rf <- predict(rf_model, newdata = test_data, type = "prob")[, "Yes"]

threshold <- 0.3
pred_adjusted <- factor(ifelse(prob_rf >= threshold, "Yes", "No"), levels = c("No", "Yes"))

conf_adjusted <- confusionMatrix(pred_adjusted, test_data$ResponseTarget, positive = "Yes")
conf_adjusted$table

cat("\nSensitivity at threshold 0.3:", round(conf_adjusted$byClass["Sensitivity"], 3))
cat("\nSpecificity at threshold 0.3:", round(conf_adjusted$byClass["Specificity"], 3))

```